

Agentic Extraction Pipeline

How it works, its steps, and how to count LLM calls

NABL Document Intelligence — backend/app/documents/. Implemented and measured 2026-09-03. Every figure in this document came from a real run; the Caveats section states plainly what has and has not been proven.

Headline: a 17-page mixed PDF went from **11–21 LLM calls to 3**. Measured average across the verification matrix: **2.25 calls per document**, max 4, none over the budget of 6.

1. What “agentic” means here

Not an autonomous agent with open-ended tool access. What was built is a **controlled state machine**: it inspects the document, decides what to do next from what it actually found, acts, checks its own output, and either finishes or takes one targeted corrective step. Every decision is made from real intermediate results rather than a fixed script, and every model call is metered against a hard budget.

The important distinction from the previous pipeline: the old one was a fixed sequence — classify, then extract per schema section, then retry per section — that spent the same calls regardless of what the document contained. The new one spends calls based on what it found.

The loop

Document → **Inspect** (per page) → **Decide** type → **Retrieve** relevant pages → **Chunk** → **Extract** → **Validate** → complete? – yes → done / no → **Recover** (targeted) → validate again → done

Deliberately, the decision layer is deterministic. An LLM is never asked to decide what to do next — only to read a document. Routing, relevance, chunking, validation and escalation are all rule-based, because they are cheap, repeatable and auditable, and because using a model to decide them would cost the very calls this work exists to save.

2. The steps

#	Step	What it does	Module	LLM?
1	Inspect	Per PAGE: native text or OCR, OCR confidence, table detection, spans, rendered image. A 17-page file is routinely 15 native + 2 scanned; the old code asked this question once for the whole document.	page_inspection.py	No
2	Classify	Scores the document type from lab vocabulary, column headers, section names, filename and result-row structure. Nova only if the score is under threshold.	classifier.py	0 or 1
3	Retrieve	Which pages are plausible evidence for the fields wanted: embedding similarity (now score-thresholded), literal label matching, and table-bearing pages.	retrieval.py	No
4	Chunk	Groups contiguous relevant pages into as few chunks as the context and the budget allow, never splitting a table away from its header.	adaptive_chunking.py	No
5	Extract	One call per chunk returning document fields, structured test rows AND the named NABL schema slots together.	combined_extraction.py	1 per chunk
6	Validate	Required fields, nulls, date sanity, duplicate tests, reversed ranges, unit/result coherence, and whether each value actually appears in the evidence.	deterministic_validation.py	No
7	Recover	ONE batched call carrying every missing/suspicious field across all chunks, with only the relevant evidence.	recovery.py	0–2
8	Vision	A page image, only for a relevant page whose OCR is the plausible culprit. Shares the recovery allowance.	recovery.py	0–2

Steps 1, 3, 4 and 6 are free. That is the design: four of the eight steps cost nothing, and they are the ones that decide how much the other four spend.

3. Where the LLM calls happen

There are exactly four call sites. Nothing else in the pipeline can reach a model.

Call site	When it fires	Count
Classification	Only when local confidence < 0.70 (configurable) AND budget allows	0 or 1
Extraction	Once per chunk produced by step 4	1 – 4
Recovery	Only if deterministic validation found something missing or suspicious	0 – 2
Vision recovery	Only if a relevant page's OCR is the plausible cause; shares the recovery allowance	0 – 2

4. How to calculate the LLM calls

The formula

$\text{total} = \text{classification} + \text{extraction} + \text{recovery}$

where each term is:

```
classification = 0 if local_confidence ≥ 0.70, else 1
extraction = min( ceil(relevant_text_chars / 12000), 4 )
recovery = 0 if validation passed, else 1 (2 only if the first
recovery still leaves gaps and budget remains)
```

Reading the extraction term

- **relevant_text_chars** is the text of the pages retrieval kept — not the whole document. Skipping irrelevant pages is where most of the saving comes from on a long file.
- **12000** is max_chunk_chars. Roughly 3,000 tokens at ~4 chars/token.
- **4** is max_initial_extraction_calls. If the relevant text needs more chunks than that, adjacent chunks are merged instead of exceeding the budget.
- Page images do not affect the chunk count — only text does. Images ride along with the pages already in the chunk.

Worked examples

Document	Relevant text	classification	extraction	recovery	TOTAL
1-page lab report, confidently classified	~1.2k chars	0	$\text{ceil}(1.2\text{k}/12\text{k})=1$	0	1
4-page born-digital certificate	~9k chars	0	1	0	1
17-page mixed PDF	~30k chars	0	$\text{ceil}(30\text{k}/12\text{k})=3$	0	3
11-page report, 273-row table, gaps found	~33k chars	0	3	1	4
Ambiguous 2-page scan	~4k chars	1	1	0	2
60-page form (budget binds)	~200k chars	0	capped at 4	2	6

The last row is the point of the cap: a document large enough to want 17 chunks gets 4, because exceeding the budget is not an option. It stops with a recorded reason rather than looping.

Reading the call log

Every document emits its own log, so the calculation never has to be inferred:

```
{ "document_id": "...", "classification_calls": 0, "extraction_calls": 3,
  "recovery_calls": 1, "vision_calls": 0, "total_llm_calls": 4,
  "input_tokens": 12480, "output_tokens": 3110, "total_tokens": 15590,
  "stop_reason": null }
```

Token counts come from Bedrock's own usage block, which the provider previously discarded — so cost is now measured, not estimated. MEASURED on a 1-page report: 1,053 input / 831 output tokens.

5. The budget

Setting	Default	Meaning
max_classification_calls	1	At most one classification call per document
max_initial_extraction_calls	4	Caps the chunk count, and therefore extraction calls
max_recovery_calls	2	Shared by text recovery AND vision recovery
max_total_llm_calls	6	Hard ceiling; the pipeline stops rather than looping
local_classification_min_confidence	0.70	At or above this, classification is free
retrieval_min_score	0.25	Below this, a page is not treated as evidence
max_chunk_chars	12000	Text budget per extraction call
ocr_low_confidence_threshold	0.75	Marks a page as possibly needing a visual check
optimized_extraction_enabled	true	false restores the previous pipeline

All are environment-overridable. Vision deliberately shares the recovery allowance: a vision call is a recovery attempt, and giving it a separate pool would let one document quietly spend double what the total implies.

6. Before and after

Document	Before	After (measured)	Change
1-page certificate	3 (classify + schema + unified)	1	−67%
4-page born-digital	3	1	−67%
17-page mixed PDF	3	3	same
Long completed application form	11 – 21 (1 per schema section + 1 retry per section + 1 unified)	≤ 6 (hard cap)	−45% to −70%
11-page report with a 273-row table	3	4 (3 extract + 1 recovery)	+1, by design

Where the savings actually came from

- **Classification became free.** It cost 0 calls on all four verification documents. The old code refused to classify locally below 5 pages, so every single-page upload — the common case — always paid for a call.
- **Three calls became one.** Classification, schema-slot extraction and open-ended extraction used to be three separate calls over the same page; they are now one request returning three named parts of one JSON object.
- **Sections stopped equalling calls.** A NABL_151 form has 10 schema sections and used to cost at least 10 calls. Sections are now grouped into at most 4 chunks.
- **Recovery became one batched call.** Gaps in three different chunks are one call carrying all of them, not three calls.

7. Measured results

Real documents, real run, 2026-09-03:

Document	Pages	Per-page method	Chunks	cls	ext	rec	Total
Chahal Mustard Oil (born-digital)	4	4 PyMuPDF	p1-4	0	1	0	1
a516f67... (mixed)	17	15 PyMuPDF / 2 OCR	p1-9, p10-13, p14-17	0	3	0	3
001_Lab-report.png (image)	1	1 OCR	p1	0	1	0	1
high-protein-paneer (273-row table)	11	9 PyMuPDF / 2 OCR	p1-3, p4-7, p8-11	0	3	1	4

Totals: 9 calls across 4 documents — average **2.25**, max **4**, none over the budget of 6. Test suite: **244 passed, 0 failed** (227 pre-existing plus 17 new, covering budget exhaustion, table detection, classification thresholds, chunk grouping, each validation rule, recovery batching and vision escalation).

8. What is NOT yet proven

This matters more than anything above, and should be read before the numbers are quoted anywhere.

- **Extraction quality is unverified.** The call counts are real. Three of the four documents returned 0 fields because Bedrock began refusing every Converse call mid-run (`ValidationException: Operation not allowed`) — including text-only calls, and including a request shape that had worked an hour earlier. That is an account/model-access condition, not a pipeline fault, but it means the run validates routing, chunking, budgeting and recovery triggering, NOT accuracy. Only 001_Lab-report.png completed (53 fields in 1 call).
- **The accuracy gate has not run.** Comparing the new path against `predictions_pre_unified/` on the full 52-document set is what would justify the claim “fewer calls at the same accuracy”. Until it does, only the first half of that claim is supported.
- **Nova is currently a hard dependency.** Every other provider rejects multi-image requests (“gemini takes one image per request, got 9 pages”), so with a page image attached to every page the fallback chain cannot serve the request. When Nova failed, one document spent 496 seconds falling through to a timing-out Ollama.
- **A 30s timeout contradicts a 65,535-token output cap.** Both are configured today. A dense 273-row table cannot be generated in 30 seconds, which is why that document has failed repeatedly regardless of this work.

Recommended next step: cap page images to the pages that actually need visual interpretation (OCR'd or low-confidence), rather than every page. That restores the fallback chain for born-digital documents, cuts payload, and matches the stated principle of “OCR first, vision only when necessary”.